THORN-2 Technical Referance Manual

aTan

May 23, 2026

Abstract

This document is a technical reference for the *Thorn*-2 (Stylised: *Þ*-2) computer. The *Þ*-2 is an RV64i compatable CPU and supporting system created largely from 74-series logic ships, with the goal of making a fully-pipelined system capable of very-high-speed execution. It also serves as a proof of [in]sanity for the creator.

Contents

List of Figures

List of Tables

Listings

1 Introduction

Thorn-2 is a RISC-V CPU, designed to be fully compatable with the RV64i (integer only) standard module as defined in the RISC-V Instruction Set Architecture (RV-ISA) specification [1]. It is a fully-pipelined design with 5parallel stages (see §Pipelining) with hazard avoidance (§Hazard Avoidance) and operand forwarding (§Operand Forwarding).

This project has some fairly nebulous goals, but the general idea is to learn how CPUs work internally and make something interesting. They are here itemised:

- Implement a RISC-V CPU
- Utilise as few non-74 series chips as possible (no FPGA)
- Design the circuitry "by hand"
- Achieve ~40 MHz operation
- Have fun :3

Please don't tell me I'm doing this inefficiently or that there article better ways to implement things - I'm well aware.

1.1 Pipelining

1.1.1 Why *Þ*-2 Differs From Classic RISC

The pipeline structure of *Þ*-2 differs from the traditional RISC one in that there are several more stages. These can be grouped into largly the same functional blocks, but spread across more stages; a setup reached after much design work on *Thorn*-1.

Classic RISC is split into five stages: Fetch, Decode, Execute, Memory and Writeback; with every instruction passing through each stage in turn (with the correct options set). In implementing this, I decided to use SRAM to store the 32 registers and to avoid using asynchronous SRAM. The latter decision was based on the false assumption that this would make the control circuitry easier to design and less error prone. I maintained the Synchronous SRAM design for quite a while, getting as far as designing the whole control circuit for the registers and starting work on the ALU, but ran into problems with timings.

The SRAM ic I had chosen - IS61NLP6432a [2] - were themselves pipelined at three stages: Assert, wait, Read/Write. Each cycle of the CPU required three register accesses (two reads and a write), which was possible to achieve if each cycle was split into four phases, $\phi_0, \phi_1, \phi_2, \phi_3$. This imposed the requirement that every pipeline stage in the CPU must take

four cycles, which was insufficient for the ALU to perform a 64-bit addition *and* perform conditional inversion of the second operand for subtraction and check conditions. Thus, I decided to go back to the drawing board and start again.

The current design of ρ-2's pipeline is based around the ALU, rather than the registers. I needed four stages to add numbers, plus two more to condition the inputs and outputs, giving six. Register accesses can be done in one step by using two copies of each register (a poor-man's dual-port RAM) which are written to simultaniously on the rising edge of the clock; then separately read on the falling edge. Now using fast, *asynchronous* SRAM instead.

1.2 Hazard Avoidance

In a pipelined system, there are frequently situations where one instruction depends on the result of its predecessor. In such a case, this results in a **Data Hazard** - what is contained in register X is out of date, and reading this will produce incorrect results.

Another hazard comes in the case that a conditional branch is dependent on the outcome of a previous operation, resulting in a **Control Hazard**. The CPU must either stall new instructions while the condition is evaluated, or speculatively execute code for either the "branch" or "no branch" condition and throw it out if needed.

1.2.1 Adding Stalls

In either hazard case, the simplest - albeit least efficient - solution is to inject NO-OPs (NOPs) after the Decode stage and only continue when there are no remaining hazards. It is, however, trivial to come up with example programs which result in more nops than executed instructions:

```
1 ;Some silly code  :3c
2 add a0, a1, a2  ;Use a0 as tally
3 add a0, a0, a3
4 add a0, a0, a4
5 add a0, a0, a5  ;Add lots of numbers in turn
6 ;...
```

When executing the above program, the CPU is forced to insert NOPs between every instruction to allow the addition result to be fully written back to `a0` before the next instruction is executed. A classic RISC-V architecture with this simple hazard avoidance would *actually* execute the code in Listing 1

This effectively quarters the useful throughput of the CPU in the worst case, though most of the time this isn't likely to occur. Using this method, the number of NOPs scales linearly with the number of stages in the pipeline,

```
;Oops, all stalls  TwT
add a0, a1, a2  ;Use a0 as tally
nop             ;Execute in ALU
nop             ;Skip memory access
nop             ;Write result to a0
add a0, a0, a3  ;Next ADD is allowed to continue
nop
nop
nop
add a0, a0, a4
nop             ;Each nop represents
nop             ;   1 stall cycle
nop
add a0, a0, a5  ;Add lots of numbers in turn
;...
```

Listing 1: Worst case scenario for consecutive adds.

so there is strong incentive to use a better system. Such a system should be able to feed data from one stage to another in order to skip some of the waiting.

1.2.2 Operand Forwarding

Operand Forwarding is one way to reduce the number of stall cycles introduced with a pipelined system. Here, in the case that an instruction needs data which has not yet been written to a register, but *is available within the pipeline*, that data can be fed forwards early so fewer stalls need to be introduced.

This method becomes very important the longer a pipeline becomes due to a greater number of cycles before the completion of each instruction: there is greater chance that a following instruction is dependent on a previous one *and* that data takes longer to become available.

Assuming the same code snipped as in §Adding Stalls, a perfect system of operand forwarding could reduce the number of NOPs to zero - i.e. what is written is executed exactly.

In *Thorn*-2 however, there is an additional complication which arises from the use of a pipelined ALU: the final result is made available in chunks across multiple stages or, should conditions need evaluation, only available after the final stage.

In fig. 1, each of the *Add* stages performs a 16-bit addition of each `int16` chunk of the 64-bit inputs. If operand forwarding were to be used, the output of *Add 1* would need to be fed back to its inputs for the next instruction. The same would be needed for each subsequent stage too. See §ALU for how this is actually done.

Invert?	Add 1	Add 2	Add 3	Add 4	Cond.
I2					
I3	I2				
I4	I3	I2			
I5	I4	I3	I2		
	I5	I4	I3	I2	
		I5	I4	I3	I2
			I5	I4	I3
				I5	I4
					I5

Figure 1: Execution flow through a pipelined ALU. I_n refers to line number in code listing.

2 Program Counter

The PC's job is (generally) to increment by 4 every clock cycle to fetch the next instruction in memory. On occasion, it will need to stall in place, load a provided value, or add a constant to its previous value to branch.

While outwardly simple, this is fairly complex to actually implement in 64 bits, expecially the branch operation which requires a 12-bit add, then a carry to propagate up the remaining bits. Since there could realistically be any arbitray sequence of branches, jumps, normal increments or stalls, all of the separate branches need to be done in parallel and selected from at the very end.

3 Decode Logic

4 Immediate Value Generator

5 Registers

RV64i specifies 32 64-bit general-purpose registers and a program counter, PC. x0 is hardwired to `0x0000 0000 0000 0000`; there is no stack pointer. None of the registers are otherwise special, but do have expected uses as per the calling convention [3], shown in table 1

As each register is effectively the same, they are most easily implemented using SRAM. Learning from *Thorn*-1, I have decided to use the IS61WV6416DBLL-10TLI-TR [4]; it's a 64k x 16-bit asynchronous SRAM ic with a 10 ns access time and single 3.3 V supply.

Table 1: RISC-V calling convention

Register	ABI Mnemonic	Meaning	Preserved across calls?
x0	zero	Zero	—(Immutable)
x1	ra	Return address	No
x2	sp	Stack Pointer	Yes
x3	gp	Global Pointer	—(Unallocatable)
x4	tp	Thread Pointer	—(Unallocatable)
x5-7	t0-2	Temp. Registers	No
x8-9	s0-s1	Callee-saved regs.	Yes
x10-17	a0-7	Argument registers	No
x18-27	s2-11	Callee-saved regs.	Yes
x28-31	t3-6	Temp. Registers	No

5.1 Theory of Operation

For every RV64i operation, there are up to two operands, meaning up to two registers may be read each cycle. There may also be a value written into a register. To avoid using multiport RAM, two copies of the registers are needed: **x** and **y**; these two are both written to and read from at the same time. The IS61WV6416is 16-bit, so there are two sets of four RAM chips total.

Both access to **x** and **y** (Read/Write) occur within one cycle; writes are completed on the rising edge, ϕ_1; and reads on the falling edge, ϕ_2. Doing it in this order means neither the writeback value, D_{WB}, nor the two read values, D_{Rs1} and D_{Rs2}, need additional latches - the former is latched from the output of the previous stage and the latter are latched on the next ϕ_1.

Some operations, such as conditional branches, compute a "result" which isn't needed. Whether such results are written to a register is tracked in the CPU-wide control logic [LINK HERE] which issues the **WRT** signal.

5.2 Control Logic

The design shown in fig. 2 shows the control logic for the register file.

The design shown in fig. 2 features two main elements which require "control logic" - the provided addresses, $\mathbf{x}_{adr}$ and $\mathbf{y}_{adr}$, and control over the data busses, $\mathbf{D}_{Rs1}$ and $\mathbf{D}_{Rs2}$. In ϕ_1, both are dedicated to the writeback operation and the read operation in ϕ_2. This occurs *whether or not* the write is needed, the only difference in the latter case being whether the IS61WV6416 commits $\mathbf{D}_{WB}$ to storage (shown in fig. 3).

In ϕ_1, if **WRT** is high, a $\overline{WE}$ controlled write cycle occurs with $\overline{OE}$ set high. This is the simplest option given the signals needed.

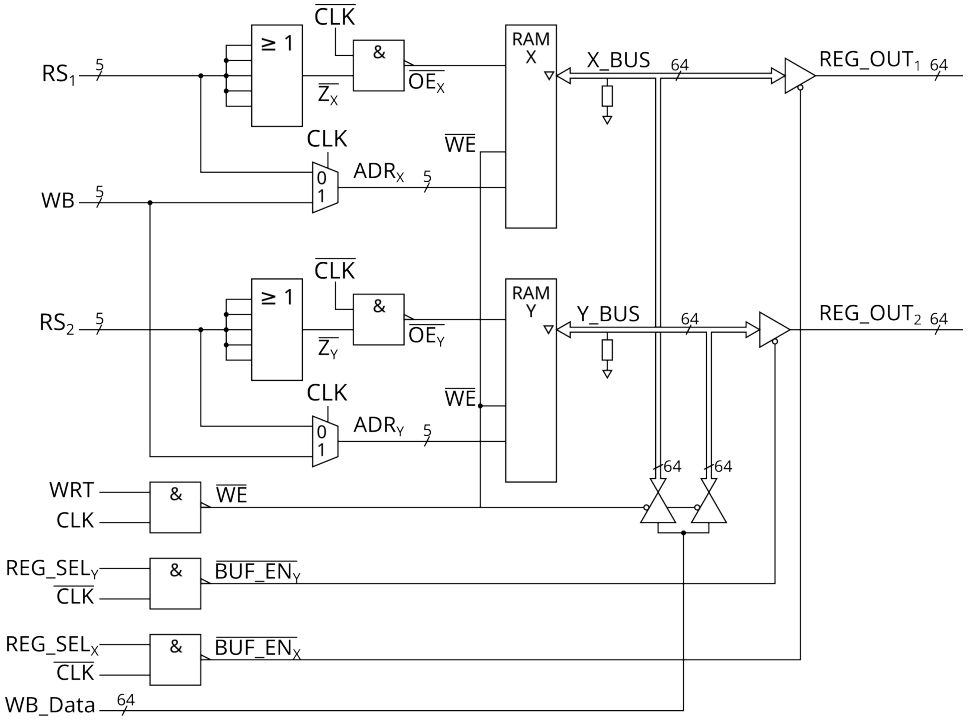

Figure 2: The *Thorn*-2 register file.

In ϕ_2, $\overline{OE}$ is set low only if the read register is *not* zero. In such a case, a register is read from as normal using an Address-controlled read cycle. When attempting to read from zero, resistors pull the data bus down, saving on parts over using buffers.

5.3 Operand Forwarding Disable

To make operand forwarding possible, the ALU may need an input from the registers, its own output, or a memory access. The registers therefore need to be able to be "selectable" by the CPU Control to pass their output on. The respective boards have buffered outputs which are enabled as needed.

6 ALU

The *Thorn*-2 ALU is comprised of three blocks - the Adder, Shifter and Logicer. They do what their name suggests: the Adder computes $A \pm B$; the Shifter shifts A up or down by n bits; and the Logic-er performs the bitwise logic operations $A \cdot B$, $A + B$ and $A \oplus B$. The 64-bit values A and B may be read from registers or immediate values derived from an instruction. Each of the three blocks are contained on separate circuit boards, but run

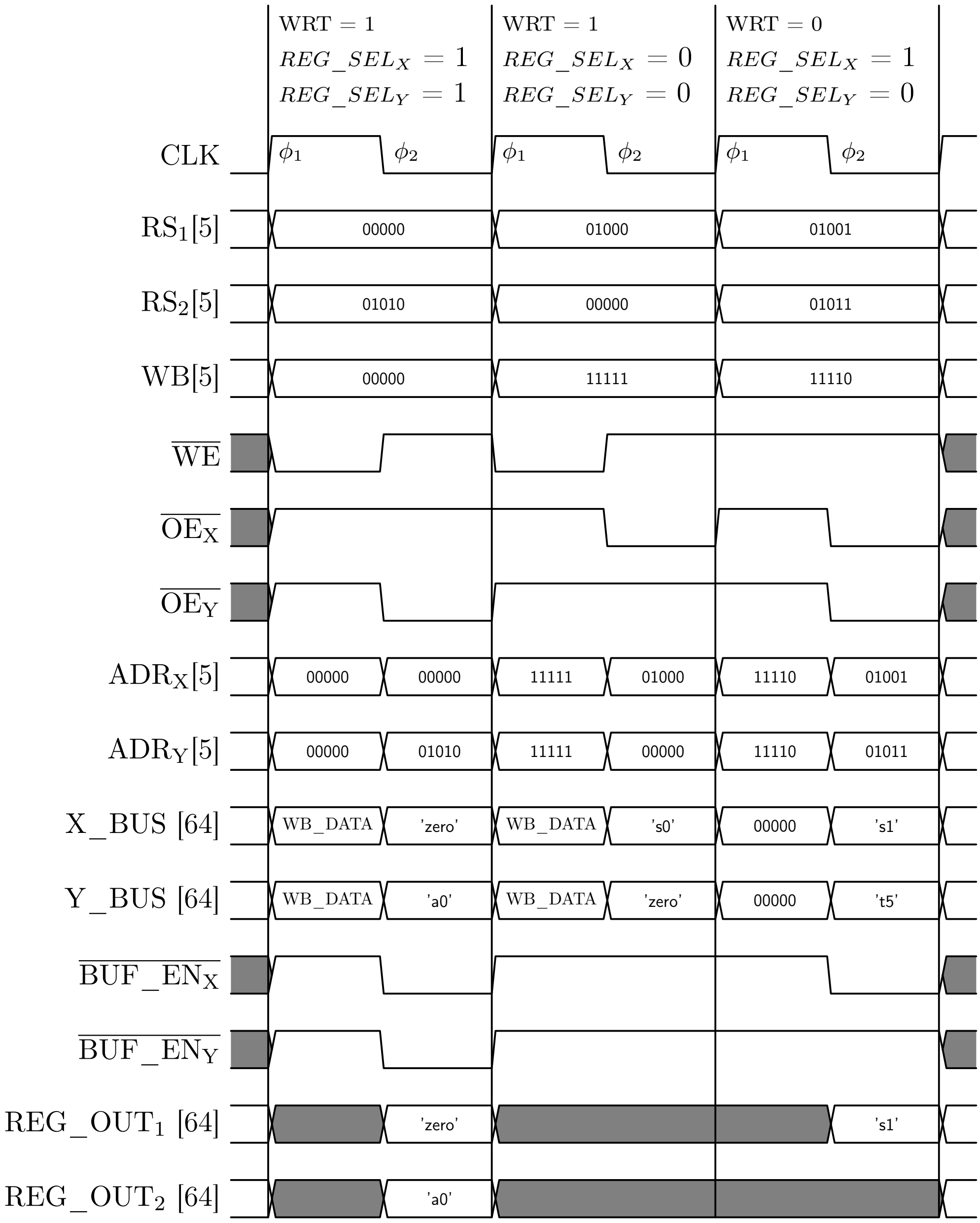

Figure 3: Register file timing diagram.

in parallel. Their outputs are calculated for all inputs with the relevant one forwarded to the next pipeline stage.

6.1 Adder

The adder computes $A \pm B$ and optionally checks the result against each of the "flags" for the calculation: $A < B$, $A = B$ and $A \geq B$. In Ƀ-1, this

procedure was to be pipelined at up to six stages, but this resulted in both huge pipelines and immense complications in feeding partial results back to previous stages, possibly from multiple instructions at multiple different points in their execution. In *P*-2, however, the entire ALU instead operates in one cycle - the length of this defining the overall cycle length of the CPU. This is also one of the few stages in the pipeline which needs non-74-series chips.

6.1.1 Method

Addition and subtraction are both done with the same circuitry, with a pre-conditioning step for the second input, **B**. Using 2's compliment to represent negative numbers, an n-bit number, x, can be negated by inverting its bits and adding 1:

$$-x := (x \oplus [2^n - 1]) + 1 \tag{1}$$

Subtracting **B** from **A** is therefore defined:

$$\mathbf{A} - \mathbf{B} \;\equiv\; \mathbf{A} + (-\mathbf{B}) \;\equiv\; \mathbf{A} + (\mathbf{B} \oplus [2^n - 1]) + 1 \tag{2}$$

Equation (2) is implemented by passing **B** through a bitwise XOR with the ADD/SUB setting, $\mathbf{K} = \overline{Add}/Sub$, and setting the carry input, C_{in} to the same:

$$\mathbf{A} - \mathbf{B} \;\equiv\; \mathbf{A} + (\mathbf{B} \oplus [\mathbf{K}]) + \mathbf{K} \tag{3}$$

The XORed **B** will be called $\mathbf{B}'$ from here.

The addition, $\mathbf{A} + \mathbf{B}' + \mathbf{K}$ is a 64-bit operation which would be much too slow if performed as a series of daisy-chained segments due to the access time of the RAM ICs. Even in the best case - where the access time were $\leq$ 8 ns - a chain of eight in a row would be a total delay of $\leq$ 64 ns, greatly bottlenecking the speed of the processor. A form of carry-lookahead is therefore used to reduce this delay.

6.1.2 Modified Carry-Lookahead

A traditional carry-lookahead circuit generates the carry signals from chunks of the inputs, ideally much faster than the same signal would be propagated up a chain of full adders. To do this, additional signals are generated or tapped from the full adder circuits:

$$X_n = A_n \cdot B_n \tag{4}$$

$$S_n = A_n \oplus B_n \tag{5}$$

These additional signals are then used to calculate whether the n-bit chunk generates, $\mathbf{G}$, or propagrates, $\mathbf{P}$, a carry. An extra signal, $\mathbf{Z}_n$ is used for checking conditions, see §Condition Checking

$$\mathbf{G} = (X_n) + (X_{n-1} \cdot S_n) + \cdots + \left(X_0 \cdot \prod_{i=1}^{n} (S_i) \right) \tag{6}$$

$$\mathbf{P} = \prod_{i=0}^{n} (S_i) \tag{7}$$

$$\mathbf{Z} = \begin{cases} 0 & , A + B \not\equiv 0 \pmod{255} \\ 1 & , A + B \equiv 0 \pmod{255} \end{cases} \tag{8}$$

A single 64Kx8 RAM chip is used as a lookup table for $\mathbf{G}$ and $\mathbf{P}$, and an additional output signals whether $\mathbf{A} + \mathbf{B} = 0$. $\mathbf{P}$ and $\mathbf{G}$ are then forwarded to another 128Kx8 RAM which stands in for the combinotorial logic which generates the carry signals.

For $n = 0$:

$$C_0 = G_0 + C_{in} \cdot P_0 \tag{9}$$

For $1 \leq n < 8$:

$$C_n = G_n + \sum_{i=1}^{n} \left(G_{n-i} \cdot \prod_{j=0}^{i-1} (P_{n-j}) \right) + C_{in} \cdot \prod_{N} (P_n) \tag{10}$$

Using this method, all eight carrys can be generated after only two RAM accesses.

6.1.3 Doing the Adding

The addition itself is "executed" in eight 128Kx8 RAM chips, each programmed at start-up from a single 128Kx8 EEPROM. The RAM is addressed by each byte of the two inputs, $\mathbf{A}_n$ and $\mathbf{B}_n$, and $\mathbf{C}_n$, which points to the result, $\mathbf{Y}_n$.

The final result is then either forwarded to the Adder's output, or changed to 0/1 for the outcome of the SLT/SLTU instructions.

In the case of ADDW/SUBW instructions which sign-extend a 32-bit answer to 64, the upper word is ignored and switched to $\mathbf{Y}[31]$.

6.1.4 Condition Checking

The RISC-V ALU needs to be able to check for three conditions for both signed and unsigned numbers: $\mathbf{A} < \mathbf{B}$, $\mathbf{A} = \mathbf{B}$ and $\mathbf{A} \geq \mathbf{B}$. These are only valid if the calculation $\mathbf{A} - \mathbf{B}$ was performed. These checks can be performed

Table 2: ALU adder EEPROM contents

$\mathbf{A}_n$	$\mathbf{B}_n$	$\mathbf{C}_n$	Address	$\mathbf{Y}_n$
0x00	0x00	0	0x00000	0x00
0x00	0x01	0	0x00001	0x01
...				
0x01	0x00	0	0x00100	0x01
...				
0x00	0x00	1	0x10000	0x01
...				
0xFF	0xFE	1	0x1FFFE	0xFE
0xFF	0xFF	1	0x1FFFF	0xFF

Table 3: Formulae for ALU result conditons

Condition	Name	Signed	Unsigned
$A < B$	LT	$\overline{A_{63} \oplus B_{63}} \oplus C_7$	$\overline{C_7}$
$A = B$	EQ	$Y = 0$	$Y = 0$
$A \geq B$	GE	$\overline{A < B}_S$	C_7

immediately after the final carry out has been calculated using the equations in table 3.

The greater/less than checks are calculated as written, but the equality check is actually calculated using the formula:

$$EQ = \prod_N \left((P_n \cdot C_{in}) + (Z_n \cdot \overline{C_{in}}) \right) \tag{11}$$

In plain english: A and B are equal if the result of $A - B$ is zero. This occures when each 8-bit chunk, Y_n, is equal to 0 with no carry in, or is equal to 255 with a carry in.

The results of the condition check are forwarded to the program counter to deal with branches.

6.2 Logicer

RISC-V specifies three bitwise logic operations: AND, OR and XOR. The most intuitive way to do this (74x08 for AND, 74x32 for OR, and 74x86 for XOR), takes 48 chips, plus some extra to choose which answer we want - quite a high number considering what this section does. This can be improved using a slightly more complex setup.

The final output, **Y**, is sent to a tri-state buffer/line driver which is only enabled when the ALU performs a logic operation.

6.2.1 Logic from MUXs

It is possible to construct any 2-input, 1-output logic function from a 4:1 MUX where the four inputs reflect the truth table of the function, and the select lines are the two inputs of the function.

Constructing the logic section this way means the control logic for this section is limited to a small ROM containing the truth tables for the MUXs.

Table 4: Truth tables of the three defined logic operations

A	B	$A \cdot B$
0	0	0
0	1	0
1	0	0
1	1	1

(a) AND truth table

A	B	$A + B$
0	0	0
0	1	1
1	0	1
1	1	1

(b) OR truth table

A	B	$A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

(c) XOR truth table

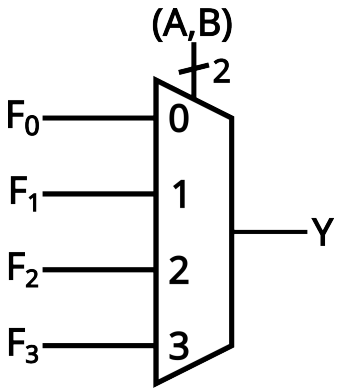

Figure 4: Wiring of a 4:1 mux as used in the logic section.

This method is preferable when considering the number of ICs used:

6.2.2 Setting up the MUX

The correct truth table is sleected using a 2:4 selector which is wired to a small, diode ROM (fig. 5). A 74x1G139 2:4 takes the two-bit function select and the desired bits are set from F_0, F_1, F_2, F_3, which are then buffered before being sent out to the MUXs. The unused decoder output is left unconnected and results in a fixed output of 0.

Table 5: Comparison of the number of components used in designs with (a) separate logic paths and (b) MUX-based logic

Section	#Gates	#ICs
AND	64	16
OR	64	16
XOR	64	16
4:1 MUX	64	32
Total	256	80

(a) Using discreet logic gates

Section	#Parts	#ICs
ROM Select	1	1
ROM Diodes	6	—
ROM Buffers	12	2
4:1 MUX	64	32
Total	75	35

(b) Using MUXs

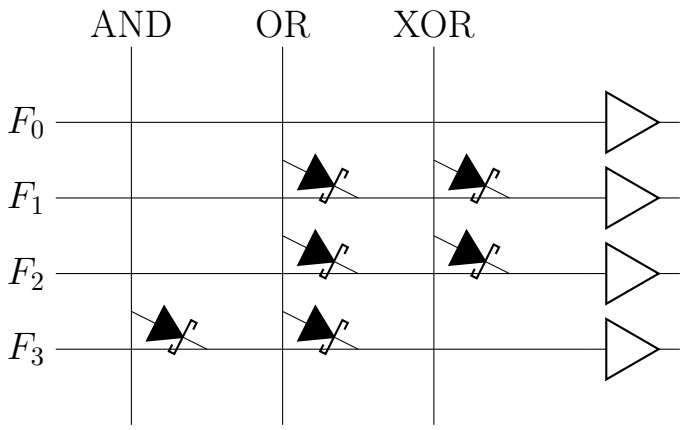

Figure 5: Diode ROM storing the truth table function, F_0, F_1, F_2, F_3. Pull down resistors on all F_n lines omitted for clarity.

As there are 64 total MUXs needed for the logic functions, the *Function* lines are buffered four times in parallel, so each buffer only needs to drive 16 inputs. The buffers are needed at all both because of the high fanout requirement and because of the forward bias of the Schottkey diodes. With a forward drop of approx. 0.3 V; this drops the nominal signal level to 3.0 V, only 0.6 V above the minimum V_{ih} of low-voltage CMOS logic chips.

6.3 Shifter

The RISC-V ISA defines several shift operations, each of which can, in one operation, shift the input by an arbitrary number of bits up to X_LEN-1. In RV64i, additional instructions shift 32-bit values and sign extend up to 64-bits. Both left and right shifts are defined, with left shifts being implemented by reversing the input, shifting, then reversing the result again:

$$A >> B := \text{REV}(\ \text{REV}(A) << B\ ) \tag{12}$$

This implementation favours left shifts, so any rightward shift will use the method in eq. (12).

Shifts are implemented using a sequential barrel shifter - a series of stages which shift the input a power of two number of places up to 32. Each stage consists of two tri-state buffers, one buffering the previous stage as is, the other buffering the previous output shifted by the next number of bits. The n^{th} stage is then controlled by the n^{th} bit of the shift amount input.

Traditionally, a sequential barrel shifter like this would use multiplexers, but this requires far too many (384 MUXs across 96 ICs) to be practical. The modified version using buffers comes from R. Baruch's RISC-V build [5], expanded to 64-bits.

Using 16-bit buffers in the 74LVC162244, the number of ICs can be reduced to 8 per stage, for a total of 48 - half that of the MUX version.

6.3.1 Bit Order Reverser

In order to shift the incoming number, **A**, rightwards, it must be bit-reversed first. This is accomplished using two tri-state buffers, one buffering the number as is, the other buffering the reversed number.

The reversal is accomplished by "simple" shuffling of the order of traces on the PCB in the folowing way:

[DIAGRAM OF PCB TRACE REVERSAL]

6.3.2 Shift Stages

References

[1] A. Waterman and K. Asanović, Eds., *The risc-v instruction set manual, volume i: User-level isa, document version 20191214-draft*, RISC-V Foundation, Dec. 2019. [Online]. Available: `https://docs.riscv.org/reference/isa/unpriv/unpriv-index.html`.

[2] I. Integrated Silicon Solution, *Is61nlp6432 ... 64k x 32 ... 2mb, pipeline 'no wait' state bus sram*, Aug. 2005. [Online]. Available: `https://www.mouser.co.uk/datasheet/3/3722/1/61NLP_NVP6436A_12818A.pdf`.

[3] K. Cheng and J. Clarke, Eds., *Risc-v abis specification, document version november 26, 2025-draft*, RISC-V International, Nov. 2025. [Online]. Available: `https://riscv-non-isa.github.io/riscv-elf-psabi-doc/`.

[4] I. Integrated Silicon Solution, *64k x 16 high speed asynchronous cmos static ram*, Mar. 2020. [Online]. Available: `https://www.mouser.co.uk/datasheet/3/3722/1/61_64WV6416DAxx_DBxx.pdf`.

[5] R. Baruch, *Lmarv-1 (tangible risc-v) part 4.1: Designing the shifter*, YouTube Video, Jul. 2018. [Online]. Available: `https://www.youtube.com/watch?v=SuUChAF3gaw`.